PCCPU Hücre Modeli

Adresler üzerinden işlem yapmanın C ile açıklanması

James Augustus

Teknik açıklama | 27 Ekim 2026

Öz

Bu metin, bir hücrenin sayı, işlem görevi veya denetleyici durumu taşıdığı küçük bir hesaplama modelini açıklar. Model C dilinde yazılmıştır. Büyük tam sayılar GMP kütüphanesiyle hesaplanır. Amaç, bir sonucun hangi hücrelerden geldiğini, bir işlemin ne zaman hazır olduğunu ve sonucun nereye yazıldığını görünür kılmaktır.

Örnek, iki hücrede 84 ve 126 sayılarını üretir. Üçüncü işçi bu değerleri toplar ve 210 sonucunu başka bir hücreye yazar. Denetleyici tamamlanan işleri sayar. Aynı düzen, 59 haneli bir sonuç için de çalıştırılır. Böylece adres düzeni ile saklanan sayının büyüklüğü arasındaki ayrım gösterilir. Son üç bölüm, bir thread kaydının simetrik iki konumu sahiplenmesi önerisini ve bunun N(p) ile ilişkisini açıklar. Bu öneri mevcut C testinin henüz uygulamadığı bir genişletmedir.

Bu çalışma bir yazılım modelidir. İşlemciler arası görev ve veri ilişkisini somutlaştırır. Fiziksel çok çekirdekli işlemci, gerçek RAM veri yolu veya hız üstünlüğü uygulaması içermez.

Okuma düzeni

1. Test ne işe yarıyor?
2. Hücre, adres ve sayı nasıl ayrılır?
3. 84 ve 126 nereden geliyor?
4. İşler hangi sırayla tamamlanıyor?
5. Büyük sayıda ne değişiyor?
6. Satır çarpımı ve N(p) ile bağlantı
7. Testlerin kapsamı ve anlamı
8. C dosyasını okumak ve çalıştırmak
9. Aynı thread içinde iki simetrik konum
10. N(p) bu tasarımın neresinde?
11. Simetriyi koruyan RAM kaydı

1. Test ne işe yarıyor?

Bir hesaplamada doğru sayıyı bulmak kadar, doğru sayıyı doğru zamanda kullanmak da önemlidir. Henüz üretilmemiş bir değeri okumak, toplamı yanlış hücreye yazmak veya bitmiş bir işi yeniden çalıştırmak sonucu bozabilir. Bu test, küçük bir düzen içinde bu sorunları kontrol eder.

Bir masada numaralandırılmış kutular bulunduğunu düşünelim. Bir kutuda sayı vardır. Başka bir kutuda “iki sayı hazır olunca bunları topla” görevi vardır. Ayrı bir kutuda ise hangi işlerin bittiğini gösteren kayıt tutulur. C programındaki hücreler bu kutuların karşılığıdır.

Bu benzetmede kutunun numarası adrestir. Kutunun içeriği değerdir. Kutunun üzerindeki “bekliyor” veya “hazır” bilgisi ise durumdur. Bu üç bilgi aynı şey değildir. Program onları ayrı tutar.

Cevaplanmak istenen soru

“Sayıları ve hesaplama görevlerini ortak bir hücre düzenine koyarsam, bağımlılıkları izleyerek doğru sonuca ulaşabilir miyim?”

Burada bağımlılık, bir işin başka bir işin sonucunu beklemesi demektir. Toplayıcı, iki sayı üretilmeden çalışamaz. Testin temel konusu bu bekleme ilişkisidir.

Bu düşüncenin somut karşılığı

Modelde bir hücre bir işçiyi temsil edebilir. Bir başka hücre sonucu saklayabilir. Bir denetleyici hücresi de tamamlanan işleri sayabilir. Böylece sayı, görev ve denetim bilgisi aynı adres düzeni içinde gösterilir.

Bu düzen, gelecekte bir çekirdeğe gönderilecek görevin nasıl tanımlanabileceğini düşünmek için kullanılabilir. Örneğin görev kaydı, kaynak hücreleri ve hedef hücreyi belirleyebilir. Ancak mevcut programda A, B ve C ayrı fiziksel çekirdekler değildir. Aynı C programının yönettiği üç işçi kaydıdır.

Testten beklenen sonuç

Bu test başarılı olduğunda, burada tanımlanan küçük modelin sınanan durumlarda doğru işlediği görülür. Bu, fikrin yazılımla sınanabilen ilk parçalarından biridir. Donanımın daha hızlı çalışacağı sonucunu tek başına vermez.

2. Hücre, adres ve sayı nasıl ayrılır?

Örnekte iki satır ve üç sütundan oluşan altı hücre vardır. Hücrelere 1000 ile 1005 arasında numaralar verilmiştir. Bu numaralar modelin kullandığı etiketlerdir.

Adres	Görev	İçerik veya iş
1000	İşçi A	İlk Pascal katsayısını hesaplar
1001	İşçi B	Yanındaki Pascal katsayısını hesaplar
1002	İşçi C	A ve B hazırsa değerlerini toplar
1003	Sonuç	Toplamı saklar
1004	Denetleyici	Tur, tamamlanan iş ve son işçi kaydı
1005	Boş hücre	Bu örnekte kullanılmaz

1000 İşçi A	**1001** İşçi B	**1002** İşçi C
1003 Sonuç	**1004** Denetleyici	**1005** Boş

Satır ve sütun numaraları sıfırdan başlar. Hücre adresi şu kuralla elde edilir:

```
adres = 1000 + 3 × satır + sütun
```

Örneğin satır 1, sütun 1 için adres 1004 olur. Bu hücre denetleyicidir. Satır 0, sütun 3 istenirse koordinat geçersizdir. Çünkü üç sütunun numaraları 0, 1 ve 2’dir.

Bir adresin üç farklı anlamla karıştırılmaması

1003, sonuç hücresinin etiketidir. 210, o hücreye yazılan sayıdır. C(10,4) ise aynı sayının Pascal üçgenindeki matematiksel konumudur. Biri depolama yerini, biri değeri, diğeri matematiksel tarifi belirtir.

Programdaki 1003 etiketi, işletim sisteminden alınmış gerçek bir RAM adresi değildir. C programı bu etiketi bir dizi elemanına çevirir. Ayrıca bu küçük tablo, tek başına tensör aritmetiği uygulaması değildir. Matris biçiminde düzenlenmiş bir kayıt alanıdır.

3. 84 ve 126 nereden geliyor?

Pascal üçgeninde her satır 1 ile başlar ve 1 ile biter. Aradaki bir sayı, bir önceki satırdaki komşu iki sayının toplamıdır. Bu nedenle bir satırdaki yan yana iki değeri toplamak, sonraki satırdaki belirli bir değeri üretir.

Bu metinde en üstteki tek 1, sıfırıncı satırdır. Satır içindeki konumlar da sıfırdan sayılır. C(n,k), n numaralı satırın k numaralı konumundaki sayıyı gösterir. Buradaki C, matematiksel katsayı gösterimidir. İşçi C adıyla veya C programlama diliyle karıştırılmamalıdır.

Dokuzuncu satır şöyledir:

```
1 9 36 84 126 126 84 36 9 1
```

Bu satırda üçüncü konum 84, dördüncü konum 126’dır. Başlangıç konumu sıfır olduğu için 84, soldan dördüncü yazılan sayıdır.

```
C(9,3) = 84
```

```
C(9,4) = 126
```

Pascal toplama kuralı bu iki değeri birleştirir:

```
C(9,3) + C(9,4) = C(10,4)
```

```
84 + 126 = 210
```

Model bunu nasıl kullanıyor?

İşçi A, C(9,3) değerini hesaplar ve 1000 adresli kendi kaydına yazar. İşçi B, C(9,4) değerini hesaplar ve 1001 adresli kendi kaydına yazar. İşçi C bu iki kaydın tamamlanmasını bekler. Sonra değerleri toplar ve 1003 adresine yazar.

Matematiksel olarak sonuç önceden C(10,4) diye tarif edilebilir. Program yine de 84 ve 126 değerlerini üretir, sonra toplar. Burada adres vermek, hesaplama maliyetini kendiliğinden ortadan kaldırmaz.

Bu örnek için komşuluk kuralı geçerlidir. Herhangi iki Pascal katsayısının toplamı aynı şekilde tek bir komşu satır konumuna indirgenemez. Adres üzerinden aritmetik tasarlanırken hangi matematiksel kuralların kullanılabildiği ayrıca belirtilmelidir.

4. İşler hangi sırayla tamamlanıyor?

Başlangıçta A, B ve C işçileri bekleme durumundadır. Sonuç hücresi boştur. Denetleyicinin tur ve tamamlanan iş sayaçları sıfırdır. Boş bir sonuç hücresini okumaya çalışmak hata olarak değerlendirilir.

Birinci tur: İki sayı hazırlanır

Program önce mevcut duruma bakar. A ve B kendi girdilerine sahiptir. Bu nedenle çalışmaya hazırdır. C ise A ve B henüz tamamlanmadığı için bekler.

Hazır işler seçildikten sonra A için 84, B için 126 hesaplanır. Değerler ilgili hücrelere yazılır ve iki işçi tamamlandı olarak işaretlenir. Denetleyicide tur sayısı 1, tamamlanan iş sayısı 2 olur. Sonuç hücresi hâlâ boştur.

İkinci tur: Toplam hazırlanır

Program duruma yeniden bakar. Artık A ve B tamamlanmıştır. C onların değerlerini okuyabilir. 84 ile 126 toplanır. 210 değeri C işçi kaydına ve 1003 adresli sonuç hücresine yazılır. Denetleyici tamamlandı durumuna geçer.

Aşama	A	B	C	Sonuç	İş sayacı
Başlangıç	Bekliyor	Bekliyor	Bekliyor	Boş	0
Tur 1 sonu	84	126	Bekliyor	Boş	2
Tur 2 sonu	84	126	210	210	3

Neden hepsi aynı turda bitmiyor?

Bir turdaki hazır işler, turun başındaki aynı durum görüntüsüne göre seçilir. Hesaplanan değişiklikler seçim bittikten sonra kaydedilir. Böylece işçilere hangi sırayla bakıldığı sonucu değiştirmez. C ilk sırada da incelense son sırada da incelense, ilk turda bekler.

Denetleyicideki 3 sayısı tamamlanan iş sayısıdır. GMP’nin yaptığı makine komutlarını saymaz. İki tur da iki fiziksel saat çevrimi anlamına gelmez. Program sıralı çalışır. Gerçek çekirdekler, kilitler veya haberleşme süreleri bu örnekte modellenmez.

`issue_address` alanı son tamamlanan işçinin adresini tutar. Donanıma bir komut gönderildiğini gösteren fiziksel bir kayıt değildir. Bitmiş modeli tekrar çalıştırmak yeni iş üretmez ve sayaçları artırmaz.

5. Büyük sayıda ne değişiyor?

İkinci gösterim aynı altı hücreyi kullanır. Yalnız hesaplanan Pascal katsayıları büyür:

```
C(199,99) + C(199,100) = C(200,100)
```

İlk iki değer eşittir. Her biri aşağıdaki 59 haneli sayıdır.

```
45274257328051640582702088538742081937252294837706668420660
```

Toplamları şudur:

```
90548514656103281165404177077484163874504589675413336841320
```

Normal, sabit genişlikli bir C tam sayısı bu değer için yeterli değildir. GMP, bir tam sayıyı gerektiği kadar bellek kullanarak saklar ve işlemleri bu gösterim üzerinde yapar. Kaynak kodda kullanılan `mpz_t`, bu büyük tam sayı türüdür.

Hücre sayısı sabit kalırken içerik büyür

Küçük örnekte de büyük örnekte de A, B, C ve sonuç hücrelerinin etiketleri aynıdır. Bağımlılık düzeni değişmez. İki üretici işi ve bir toplama işi vardır. Mantıksal tur sayısı yine 2, tamamlanan iş sayısı yine 3 olur.

Ancak bu, iki örneğin aynı süreyi veya aynı belleği kullandığı anlamına gelmez. Bir işin içindeki aritmetik, sayı büyüdükçe daha fazla çalışma gerektirebilir. GMP'nin ayırdığı alan da büyüyebilir. Test zaman veya bellek karşılaştırması yapmaz.

Matematiksel tarif ile açılmış değer

C(200,100), sayının kısa bir tarifidir. Program bu tarifi kullanarak sayının bütün değerini hesaplar. Dolayısıyla örnek, dev bir sayının tüm değerini hiç üretmeden işleme yöntemini henüz uygulamaz.

Bazı işlemler matematiksel tarifler üzerinde yürütülebilir. Bunun için hangi işlemin hangi tarifi ürettiği ve hangi aşamada sayının açılması gerektiği belirlenmelidir. Bu model, böyle bir yapıda görevlerin ve sonuçların yerini göstermek için bir başlangıçtır.

6. Satır çarpımı ve N(p) ile bağlantı

Pascal satırları, uygun tabanlarda bir sayının haneleri olarak da görülebilir. b tabanı için R(b,n) = (b+1)^n yazalım. Kuvvetler çarpılırken üsler toplandığından şu eşitlik elde edilir:

```
R(b,r) × R(b,s) = R(b,r+s)
```

Bu değer eşitliği geçerlidir. Ancak sonuç satırındaki bütün Pascal katsayılarının doğrudan birer hane olarak okunabilmesi için katsayılar tabandan küçük olmalıdır. En büyük katsayı ortada bulunduğundan, orta katsayıyı kontrol etmek yeterlidir.

Eldesiz örnek

Onluk tabanda ikinci satır 1, 2, 1 katsayılarından oluşur. $11^2 = 121$'dir. Bu sayıyı kendisiyle çarpınca $11^4 = 14641$ bulunur. Dördüncü satırın katsayıları 1, 4, 6, 4, 1'dir. En büyük katsayı 6, taban olan 10'dan küçüktür. Katsayılar doğrudan hane olarak görünür.

Elde gerektiren örnek

$1331 \times 121 = 11^3 \times 11^2 = 11^5 = 161051$ olur. Beşinci satırın katsayıları 1, 5, 10, 10, 5, 1'dir. 10 katsayısı onluk tabanda tek hane olamaz. Elde taşıma gerekir. Bu nedenle sonuç yazısındaki haneler artık katsayıların yalın dizisi değildir.

C testindeki kontrol

`row_product` yordamı orta katsayıyı hesaplar ve tabanla karşılaştırır. Küçükse eldesiz sonucu verir. Eşitlik yeterli değildir. Örneğin taban 3 ve satır 3 için orta katsayı 3 olduğundan elde gerekir.

Bu yordam satırları bütünüyle açıp çarpmaz. Yalnız birleşen satırın eldesiz olup olmadığını sınar. Terminaldeki 121 ve 1331 çarpım cümleleri açıklama satırlarıdır. Otomatik testte denetlenen bölüm, ilgili elde sınırı kararıdır.

Mersenne tabanı b = 2^p - 1 seçildiğinde N(p), son eldesiz satırın numarasıdır. Bu durumda $r+s \le N(p)$ koşulu kullanılabilir. Buradaki C modeli N(p)'yi hesaplamaz. Ayrıca N(p) matematiksel işlevi ile önerilen çok parametreli hücre adresi gösterimi aynı tanım değildir. Çok parametreli gösterim bu kodda kesinleştirilmemiştir.

7. Testlerin kapsamı ve anlamı

Testler yalnız terminale 210 yazıldığını kontrol etmez. Sonuca giden düzeni ve bazı yanlış kullanım durumlarını da sınar. Bir koşul sağlanmazsa program hata bildirir ve başarısız çıkış koduyla durur.

Kontrol	Sorulan soru
Hücre adresleri	Altı hücrenin adresleri ayrı ve sınırlar doğru mu?
Geçersiz istek	Uygun olmayan satır veya işçi sırası reddediliyor mu?
Bağımlılık	C, iki üretici tamamlanmadan bekliyor mu?
Altı işçi sırası	A, B ve C’nin inceleme sırası sonucu değiştiriyor mu?
Tekrar çalıştırma	Bitmiş işler ikinci kez sayılıyor mu?
Pascal karşılaştırması	Hücrelerin toplamı bağımsız hesapla uyuşuyor mu?
Elde sınırı	Katsayının tabana eşit olması da reddediliyor mu?

20.100 toplam nasıl sınanıyor?

Birinci satırdan iki yüzüncü satıra kadar, her satırın bütün komşu katsayı çiftleri kullanılır. Birinci satırda 1 çift, ikinci satırda 2 çift, iki yüzüncü satırda 200 çift vardır. Toplam çift sayısı 1 + 2 + … + 200 = 20.100’dür.

Model katsayıları GMP’nin binom yordamıyla üretir. Karşılaştırma tarafı ise Pascal satırlarını yalnız önceki satırdaki komşu değerleri toplayarak kurar. Böylece beklenen sonucu üretmek için aynı binom yordamı yeniden çağrılmaz. İki yol da büyük tam sayıları saklamak için GMP kullanır. Karşılaştırma, ayrı bir aritmetik kütüphanesiyle yapılmış değildir.

Her çiftte kurulum, yürütme ve sonuç eşitliği kontrol edilir. Bunlara adres, durum ve sınır kontrolleri eklenince toplam 60.353 koşul değerlendirilir. Bu sayı 60.353 farklı matematiksel teorem veya farklı işlemci deneyi anlamına gelmez.

Başarılı sonuç nasıl okunmalı?

Bu paketin test çalıştırmasında bütün kontroller geçmiştir. Böylece sınanan aralıkta hesap doğruluğu, sıra bağımsızlığı ve hazır olma davranışı desteklenir. Bütün olası büyüklükler için biçimsel doğruluk kanıtı, bellek yetersizliği testi veya gerçek eşzamanlı çekirdek testi yapılmış sayılmaz. Donanım hızına ve enerji tüketimine ilişkin ölçüm yoktur.

Bu çalıştırmanın sonuç özeti ve ham çıktısı ayrıca verilmiştir.

8. C dosyasını okumak ve çalıştırmak

Paket, anlatım ile çalışan örneği birlikte içerir. Aşağıdaki yollar depo köküne göredir.

Dosya	İşlev
kod/PCCPU/pccpu_address_model.c	Model, testler ve iki sayısal gösterim
kod/PCCPU/calistir.sh	Derler, testleri ve gösterimi çalıştırır
tr/md/CTCPU_PARCALAR/PCCPU/PCCPU_C_TESTI_ACIKLAMA.md	Açıklamanın düzenlenebilir metni
tr/pdf/CTCPU_PARCALAR/PCCPU/PCCPU_C_TESTI_ACIKLAMA.pdf	Bu açıklamanın PDF sürümü
README.md	Kısa başlangıç bilgisi

Tek komutla çalıştırma

C11 destekleyen bir C derleyicisi, GMP geliştirme dosyaları ve Bash gerekir. Depo kökündeki terminalde şu komut kullanılır:

```
bash kod/PCCPU/calistir.sh
```

Betik önce C dosyasını derler. Ardından `--test` ile kontrolleri, `--demo` ile örnekleri çalıştırır. Derlenmiş program geçici alanda tutulur ve betik bitince kaldırılır. Paket içindeki kaynak dosyaları değişmez. Son satırlarda 210 ve büyük sayı sonucu görülür.

Kaynakta hangi bölüm ne yapıyor?

`Cell`, bir hücrenin değerini ve durum alanlarını tanımlar. `Model`, altı hücreyi ve Pascal girdilerini bir araya getirir. `init` başlangıç durumunu kurar. `step` bir mantıksal turu yürütür. `run` sonuç hazır olana kadar ilerler. `read_value` yalnız okunmaya hazır sayı hücrelerine erişir. `clear`, GMP için ayrılan değerleri serbest bırakır.

`tests` otomatik kontrolleri yürütür. `demo` küçük ve büyük örneği ekrana yazar. `main`, hangi çalışma biçiminin seçildiğini belirler. Örnek girdileri kaynakta sabittir. Program henüz kullanıcıdan serbest biçimde sayı veya hücre görevi alan bir komut sistemi değildir.

Modelin geliştirilmesinde sonraki somut adım

Bugün C işçisinin kaynakları ve hedefi kod içinde sabittir. Bir sonraki adım, işlem türünü, kaynak adreslerini ve hedef adresini ayrı bir görev kaydına taşımaktır. Böylece aynı yürütme düzeni farklı görevleri okuyabilir. Birden fazla işçinin aynı hedefe yazması gibi durumlar için de açık kurallar gerekir. Bu adım, mevcut örneğin üzerine kurulacak ayrı bir geliştirmedir.

Kaynak ve kullanım

Bu açıklama paketteki C kaynağını ve onun testlerini esas alır. Kod MIT lisansıyla, bu metin CC BY 4.0 lisansıyla sunulur. Telif hakkı © 2026 James Augustus.

9. Aynı thread içinde iki simetrik konum

Bir kutunun iki bölmesi olduğunu düşünelim. Kutunun kimliği aynıdır ama bir bölmesinde sayının sağ tarafına, diğerinde sol tarafına ait bilgi tutulur. Önerilen thread kaydı da böyle çalışabilir. Thread kimliği tek olur. Temsil ettiği konumlar iki tane olabilir.

Dördüncü Pascal satırı [1,4,6,4,1]'dir. Kuvvet konumlarını sağdan sayarsak uçlar x^0 ile x^4, bir içteki çift x^1 ile x^3, merkez ise x^2 olur.

Kayıt	Ortak katsayı	Taraf 0	Taraf 1
thread_no_1	1	x^0	x^4
thread_no_2	4	x^1	x^3
thread_no_3	6	x^2	Aynı merkez

Aynı olan ve farklı olan

İki uçta da katsayı 1'dir. Ama x^0 ve x^4 genel olarak farklıdır. x = 7 seçilirse birinci değer 1, ikinci değer 2401 olur. Tek thread bu iki değeri iki ayrı alanında tutabilir. Ortak katsayı ayrıca bir kez saklanabilir.

Bir içteki çiftte ortak katsayı 4'tür. Kuvvet değerleri 7 ve 343'tür. Sayıya yaptıkları katkılar ise 4 × 7 = 28 ve 4 × 343 = 1372'dir. Katsayı, kuvvet değeri ve katsayıyla çarpılmış katkı ayrı büyüklüklerdir. Burada 4^1 veya 4^3 hesaplanmaz.

Tarafı seçmek

Öneri olarak `T(n,k)[a]` yazalım. n, Pascal satırının numarasıdır. k, sağdan sayılan ilk üs konumudur. $0 \le k \le \lfloor n/2 \rfloor$ alınır. a = 0 için k, a = 1 için n-k seçilir. Böylece `T(4,0)[0]` x^0'ı, `T(4,0)[1]` x^4'ü seçer.

Bu gösterim yalnız hangi tarafın seçildiğini söyler. Katsayının mı, kuvvet değerinin mi, yoksa ağırlıklı katkının mı okunacağı işlem türüyle belirtilmelidir. Bu bir öneridir. N(p)'nin anlamını değiştirmez ve henüz kesinleşmemiş çok parametreli N gösterimini sabitlemez.

Merkez nasıl korunur?

n = 4, k = 2 olduğunda k = n-k olur. İki taraf aynı merkez konumudur. $6 \times 7^2 = 294$ katkısı yalnız bir kez eklenir. Toplam 1 + 2401 + 28 + 1372 + 294 = 4096'dır. n = 3 satırında ise (0,3) ve (1,2) çiftleri vardır. Ortadaki iki katsayı eşit olsa da konumları farklıdır ve ikisi de hesaba katılır.

10. N(p) bu tasarımın neresinde?

Simetrik kayıt fikrinin ilk dayanağı Pascal katsayılarının simetrisidir:

```
C(n,k) = C(n,n-k)
```

Bu eşitlik bir katsayıyı iki konuma bağlamayı sağlar. İkinci dayanak, aynı katsayıları taban ağırlıklarıyla birleştiren binom açılımıdır:

```
(b+1)^n = Σ C(n,k) b^k, k = 0,...,n
```

Her katsayının sayının bir hanesi olarak okunabilmesi için tabandan küçük olması gerekir. En büyük katsayı c(n) = C(n,⌊n/2⌋) olduğundan, bütün satır için c(n) < b koşulu yeterli ve gereklidir.

⌊n/2⌋ gösterimi, n/2 değerinin aşağı yuvarlanmış tamsayı kısmıdır.

N(p) tanımı

p ≥ 2 bir tamsayı ve b = 2^p - 1 olsun. Bu tabanda son eldesiz Pascal satırının numarası

```
N(p) = max { n ≥ 0 : C(n,⌊n/2⌋) < 2^p - 1 }
```

olarak tanımlanır. Temiz satırlar başlangıçtan itibaren 0, 1, …, N(p) aralığını oluşturur. Bunun nedeni merkez katsayısının satır ilerledikçe azalmamasıdır. Dolayısıyla n ≤ N(p) ise katsayı simetrisi doğrudan hane simetrisi olarak kullanılabilir.

p = 3 için b = 7’dir. Dördüncü satırın merkezi 6 < 7, beşinci satırın merkezi 10 ≥ 7 olur. Bu yüzden N(3) = 4’tür. Dördüncü satırdaki iki uç ve iç çiftler, 4096 sayısının 7 tabanındaki simetrik hanelerini temsil eder.

Sınır aşılırsa ne olur?

Beşinci satırın katsayıları [1,5,10,10,5,1] olarak hâlâ simetriktir. Fakat 10, 7 tabanında tek hane değildir. Elde taşıma sonunda 32768 sayısının haneleri [1,6,4,3,5,1] olur. Bunlar simetrik değildir. RAM kaydı katsayıları tutuyorsa eşleştirme korunabilir. Son sayının normalleştirilmiş hanelerini tutuyorsa aynı eşleştirme otomatik kullanılamaz.

Önceki 84 + 126 = 210 örneği de bu ayrımı gösterir. Tamsayı toplamı doğrudur. Ancak taban 127 ise N(7) = 9 olduğu için sonuç satırı 10 eldesiz değildir. Tabanı 255 seçersek N(8) = 10 olur ve onuncu satırın en büyük katsayısı 252 tabana sığar.

Pascal çalışmasına bağlantı

Pascal araştırması bu taban gösteriminin matematiksel çalışmasıdır. Yayımdaki Türkçe not, eldesiz koşulu ve belirli satır seçimini açıklar. Burada tanımı verilen N(p), aynı koşulun geçerli olduğu son satırı belirtir. N(p) bir zamanlayıcı veya RAM devresi değildir. Bu tasarımda katsayıların ne zaman doğrudan hane sayılabileceğini belirleyen matematiksel sınırdır.

11. Simetriyi koruyan RAM kaydı

Bir thread kaydında iki simetrik tarafın bilgisini birlikte tutmak için aşağıdaki alanlar düşünülebilir. Bu tablo önerilen mantıksal kaydı gösterir. C programındaki mevcut Cell yapısına henüz eklenmiş değildir.

Alan	Tutulan bilgi
n, k	Satır ve çiftin ilk üs konumu
x veya x_ref	Kuvvet tabanı ya da onun adresi
coefficient	İki tarafa ait ortak Pascal katsayısı
power[0], power[1]	Hesaplanmışsa iki ayrı kuvvet değeri
ready[0], ready[1]	Her değerin hazır olup olmadığı
owner	Kaydı sahiplenen thread kimliği

Değerleri tutmak veya tariflerini tutmak

Birinci tercih, x^k ve x^(n-k) değerlerini hesaplayıp iki alanda saklamaktır. Kullanıcı iki tarafa da hazır değer olarak erişir. İkinci tercih, n, k, x ve katsayıyı saklayıp istenen değeri daha sonra hesaplamaktır. Bu durumda saklanan şey hesaplanmış iki sayı değil, onları üretecek tariftir.

Simetrik bir Pascal satırında ⌊n/2⌋ + 1 farklı katsayı yeterlidir. Ancak iki kuvvetin tümü, adresler ve durum bilgileri de saklanıyorsa toplam belleğin yarıya ineceği söylenemez. Gerçek alan kazancı kayıt biçimi seçilip ölçüldükten sonra belirlenebilir.

Yazma ve geçerlilik kuralı

x, n veya k değişince eski kuvvet değerleri geçersiz olur. Bir tarafın katsayısı diğerinden bağımsız değiştirilecekse ortak katsayı bağlantısı ayrılmalıdır. Yoksa simetri varsayımı artık doğru olmayan bir eşitlik dayatır. İki ayrı taraf için hazır olma durumu da ayrı izlenmelidir.

Merkez tek konum olduğunda iki seçici aynı kayda yönlenebilir. Toplama merkez değerini bir kez kullanmalıdır. Böylece iki taraflı adresleme, tek merkez değerini yanlışlıkla iki kez saymaz.

PCCPU ile bağlantısı

Thread kimliği işin sahibini, RAM adresi kaydın yerini, taraf seçimi ise kayıttaki konumu belirtir. PCCPU bu görevleri başlatıp tamamlanma durumlarını izleyebilir. Bir thread’in iki değer taşıması tek başına iki işlemi aynı anda yaptığı anlamına gelmez.

Sıradan RAM üzerinde bu kayıt düzeni kurulabilir. Özel RAM donanımı için hücre genişliği, erişim yolları ve yazma kuralları ayrıca tasarlanmalıdır. Mevcut C testindeki 60.353 kontrol, üretici ve toplayıcı modeline aittir. Bu yeni simetrik RAM tasarımının uygulanmış veya sınanmış olduğunu göstermez.